Security Review

Cantina Managed

COINBASE: CUSTOM-STABLECOIN

Cantina Managed review by:
Slowfi, Lead Security Researcher
Jiri123, Security Researcher

CANTINA

August 14, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Jul 23rd to Jul 28th the Cantina team conducted a review of custom-stablecoin on commit hash cba1c33c. The team identified a total of **6** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	6	5	1
Total	**6**	**5**	**1**

2.1 Scope

The security review had the following components in scope for custom-stablecoin on commit hash cba1c33c:

```
solana/programs/mint-controller/src
├── constants.rs
├── errors.rs
├── lib.rs
├── state.rs
└── utils.rs
```

3 Findings

3.1 Informational

3.1.1 Canonical IDL initialization can be front run

Severity: Informational.

Context: Cargo.toml#L10-L17.

Description: The program declares Anchor's `no-idl` feature but does not enable it in the default production feature set. Anchor `0.31.1` therefore adds a legacy instruction namespace to the compiled program for creating, writing, replacing, transferring, and closing a canonical IDL account.

The generated `Create` handler accepts any funded `from` signer. It verifies that the supplied executable program is the current program, creates the canonical IDL address, and stores the first signer's key as its authority. It does not require the program data upgrade authority, the Mint Controller global admin, or the deployer.

This path was reproduced against the exact reviewed ELF. An unrelated signer created the canonical IDL account, wrote synthetic compressed JSON, and made the locked Anchor client's `Program.fetchIdl` return it. A later legitimate initializer failed because the account already existed, and its write failed the generated `has_one = authority` constraint. `Program.at` then accepted the forged IDL's embedded address and constructed a client for an attacker selected program ID.

The attacker cannot modify Mint Controller state, replace executable code, or mint tokens through this path. However, wallets, explorers, decoders, account resolvers, or signing tools that trust the canonical IDL can display or construct an attacker selected schema. No production consumer in this repository was shown to rely on this path, so the issue is informational.

Recommendation: Enable `no-idl` in the crate's default feature set so the production artifact cannot include the legacy namespace because a deployment command omitted a build flag. Continue using `idl-build` to generate the client side IDL, and publish it as a hash attested release artifact through `inventory_metadata.source_code_data`.

Add a release check that builds the exact production feature set and verifies that the legacy IDL `Create` payload is rejected with `IdlInstructionStub`. This preserves Anchor client and TypeScript IDL generation without leaving the canonical IDL account available to the first caller.

The program is not deployed yet, so this can be corrected before any canonical IDL account exists. If that assumption changes, inspect the account, authority, and content before enabling `no-idl`, because removing the handlers does not remove or recover an existing account.

Coinbase: Fixed in commit cc76d79d.

Cantina Managed: Commit cc76d79d enables `no-idl` in the default feature set, making the default build byte-identical to a `--features no-idl` build. The accompanying release check asserts that `anchor:idl` is absent from the built artifact by testing for the string produced by `IdlAccount::seed()`, which is compiled out when `no-idl` is active.

3.1.2 Negative tests can catch their own failure assertions

Severity: Informational.

Context: mint-controller.ts#L223-L246.

Description: Most negative tests put `assert.fail("expected <error>")` inside the same `try` block whose `catch` checks the thrown string for `<error>`. If the RPC unexpectedly succeeds, Chai throws a local assertion error containing the expected error name, and the `catch` block can accept that local assertion as though it came from the Solana program.

The issue was reproduced with the checked in Chai dependency:

```
AssertionError: expected InvalidMintAuthority
catch assertion PASSED
```

The affected fail sites include lines `243`, `279`, `407`, `423`, `465`, `482`, `496`, `510`, `523`, `535`, `605`, `642`, `675`, `727`, `742`, `783`, `847`, `878`, `941`, `972`, `1048`, and `1103`. The later exact balance assertions in the cases containing lines `642` and `1048` still detect an unexpected successful mint, but their local error oracle accepts the wrong exception and cannot prove the expected program error. The other twenty cases lack an equivalent postcondition.

These tests appear to cover authorization, validation, allowlist, pause, and rate limit failures, but they can report success without observing the expected program rejection. A future regression could therefore pass the checked in test suite.

Recommendation: Replace all 25 `assert.fail` sites with a rejection helper that catches only errors thrown by the awaited RPC promise, throws outside the `catch` path if the promise resolves, and asserts the exact Anchor error code rather than broad message substrings. Rerun every authorization, validation, allowlist, pause, and rate limit negative test after making the change.

Solana rolls back all writes when a transaction fails, so a complete account snapshot is not necessary in every administrative negative test. Add exact supply, destination balance, and rate limit capacity postconditions to the mint path negatives, where an unexpected success would move value. Keep broader rollback snapshots for focused CPI failure and multiple instruction atomicity tests.

Coinbase: Fixed in commit 6fdc80c1.

Cantina Managed: Commit 6fdc80c1 replaces all 25 `assert.fail` sites with a rejection helper that validates only errors thrown by the awaited RPC promise. Supply, balance, and capacity postconditions were added to the mint-path negatives. The fix also surfaced a misattributed error code: the `has_one` constraint was being overridden by `@MintControllerError::Unauthorized` annotations, causing it to surface as `Unauthorized` rather than `ConstraintHasOne`, which had been hidden by the loose regex match.

3.1.3 README misstates allowlist authority and lifecycle

Severity: Informational.

Context: README.md#L18-L41.

Description: The README says the per mint `admin` manages recipient allowlists and lists `admin` as the caller for `add_allowed_mint_recipient` and `remove_allowed_mint_recipient`. The code enforces `allowlist_authority` for those instructions through the `AddAllowedMintRecipient` and `RemoveAllowedMintRecipient` account contexts.

The README also says `add_allowed_mint_recipient` lazily creates the allowlist, while the implementation creates the allowlist in `configure_minter` via `init_if_needed`. Finally, the preflight sequence documents `initialize(admin, rate_limit_authority)` even though the instruction also requires `allowlist_authority`.

Operators can assign or rotate the wrong keys and then fail to execute allowlist operations during deployment or incident response. This is a documentation and operator guidance mismatch rather than an authorization bypass in the program.

Recommendation: Correct the README sections that describe the allowlist role and lifecycle:

- Add `allowlist_authority` to the roles table and to the documented `MintRoles` PDA fields.
- Document `update_allowlist_authority` in the instruction table.
- State that `allowlist_authority`, rather than the per mint `admin`, controls add and remove allowlist operations.

- State that `configure_minter` creates both the rate limit and allowlist PDAs, and that `revoke_minter` always closes both.
- Update the initialization signature and deployment preflight to include `admin`, `rate_limit_authority`, and `allowlist_authority`.

Coinbase: Fixed in commit f12dcd7f.

Cantina Managed: Commit f12dcd7f corrects all five originally identified items and four additional instances of the same documentation drift: a missing `allowlist_authority` row, no `update_allowlist_authority` entry, `MintRoles` listed with only two keys, and `revoke_minter` described with an "if present" qualifier. The initialization signature and preflight section were updated as part of the fix for Finding 5.

3.1.4 Mint authority handoff occurs before controller setup

Severity: Informational.

Context: lib.rs#L77-L84.

Description: The README instructs operators to transfer SPL mint authority to the program's `mint_authority` PDA before calling `initialize`. The program requires that order because `initialize` rejects a mint unless its current mint authority is already the PDA.

After the authority transfer, only the program can sign for that PDA. The program exposes `mint_to` through `mint_tokens`, but it does not expose a governed SPL Token `set_authority` CPI that can return or rotate mint authority. If initialization cannot complete because of a wrong program ID, unsupported token program, unavailable global admin, incorrect build artifact, or another deployment issue, there is no source level recovery path for the mint authority. Recovery then depends on the upgrade authority and a new binary if the program remains upgradeable.

An onboarding failure after the handoff can leave issuance unavailable for the affected mint because the original authority can no longer recover the mint. This does not let an unauthorized account mint tokens.

Recommendation: Adopt the proposed prepare then handoff deployment order. Remove the mint authority requirement from `initialize`, then initialize the mint roles, configure its minters, populate the allowlists, and prove that the critical role keys can sign while the original authority still controls the mint.

Before the final handoff, verify the exact deployed program address and binary hash, derive the `mint_authority` PDA from that deployed address, and confirm that the mint is owned by the classic SPL Token program. Make `spl-token authorize` the final deployment step. Retain the existing mint authority constraint in `MintTokens`, so controller minting remains disabled until the handoff succeeds.

Add integration tests proving that `mint_tokens` rejects before the handoff, failed setup leaves the original mint authority recoverable, and the final authority transfer enables minting through the controller. A permanent decommission instruction is not required if the deployment pipeline enforces these checks and this ordering.

Coinbase: Fixed in commit f669314a.

Cantina Managed: Commit f669314a adopts the prepare-then-handoff deployment order: the mint authority requirement is removed from `initialize`, and `spl-token authorize` becomes the final deployment step. The existing authority constraint in `MintTokens` is retained, and three integration tests are added. An additional build-time guard was introduced to prevent `initialize_global` from being called with the `INITIAL_GLOBAL_ADMIN` placeholder; `anchor build` now requires `--features localnet` until a real key is set.

3.1.5 Single-step global admin rotation has no recovery path

Severity: Informational.

Context: lib.rs#L42-L46.

Description: `update_global_admin` overwrites `admin` in one call, checking only that the new key is non-zero. A wrong but valid key takes control immediately, and since `GlobalConfig` cannot be closed or re-initialized, recovery needs a program upgrade.

Recommendation: `update_global_admin` sets a `pending_admin`, and a new `accept_global_admin` signed by that key completes the transfer. Same as `Ownable2Step` on the EVM side.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged. A co-signed transfer is not feasible with the current tooling, which supports only one cold key signer per transaction (additional signers must be private key files on disk). Rather than adding program state for a propose-and-accept pattern, the deployment pipeline enforces that rotation waits on an observed signature from the incoming key on the target cluster, ensuring an unsignable or wrong-network key is rejected before the swap.

3.1.6 Rent funding expectation for minter PDAs is undocumented

Severity: Informational.

Context: lib.rs#L499.

Description: `ConfigureMinter` sets `payer = rate_limit_authority` for both PDAs, while `RevokeMinter` uses `close = admin`. When the roles are separate keys, about 0.0247 SOL per minter moves from one to the other each grant/revoke cycle.

Recommendation: Document in the README preflight that `rate_limit_authority` pays the rent and `admin` reclaims it, and that rent goes to whoever holds the `admin` role at close time rather than the original payer.

Coinbase: Fixed in commit 6ef30d52.

Cantina Managed: Commit 6ef30d52 adds documentation of the rent flow under a `### Rent` sub-section in the PDAs section of the README, positioned alongside the PDAs it describes.